

Reference Pack

Companion to the Meridian Insights project brief. This is the reference detail — the source and pipeline expectations, the field guide, the fifteen questions, and how to package what you send back. Read the brief first; this fills in the specifics.

0. How the work breaks down

Three parts, meant to fit together as one system:

1. **Source & medallion pipeline** — locate the public source, build the pipeline that lands the raw extract in **bronze/**, conforms it in **silver/**, and serves analysis-ready tables from **gold/** (details in §1). This is the foundation everything else stands on — sound and auditable, but kept in proportion.
2. **The narrative assistant (RAG)** — semantic search over the complaint narratives that works *together with* metadata filters (company, product, state, date), answers with Complaint-ID citations, and refuses honestly when the data can't support an answer.
3. **The natural-language query harness** — let an analyst ask the *structured* data a question in plain English ("*how many...*", "*which company...*", "*what's the timely-response rate for...*") and get a straight answer back, computed by a **real query your system generates and runs read-only against your gold tables** — with the generated query **returned alongside the answer** so it can be inspected and trusted. This harness is the analytical backbone: the assistant's "a number *and* the story" questions should get their numbers from it and their narrative colour from Part 2.

The fifteen questions below exercise all three: the narrative-only ones go through Part 2, the counting/ranking/trend ones through Part 3, and the "spike... and what was driving it" ones through both.

1. The source and the pipeline

The source

The **CFPB Consumer Complaint Database** — the US Consumer Financial Protection Bureau's public archive of consumer-finance complaints. It is openly published and accessible in more than one way (bulk download and a public API, among others); **finding it, choosing an access path, and justifying that choice is part of the exercise**. The database updates daily.

Pinned extract (so everyone works the same population): complaints **received 2024-01-01 through 2025-12-31** that **have a consumer narrative**. Expect on the order of **two million records**. Pull once, snapshot into bronze, record the pull date — all your numbers should be reproducible from that snapshot.

The medallion layers (you can use local folders on your machine to demonstrate these layers)

You don't need a real cloud storage account or a warehouse to satisfy this — three folders on your own disk, named `bronze/` , `silver/` , `gold/` , with your code reading from one and writing the next, demonstrates the pattern just as well as an S3 bucket would. What we're grading is the *separation of concerns and the audit trail*, not the infrastructure.

Layer	What lives there	What we look for
<code>bronze/</code>	The raw extract exactly as fetched — no edits, no filtering beyond the pinned extract. Plus a manifest : source URL/endpoint, pull timestamp, method, record count, file checksums.	Immutability and an audit trail. If a number is ever doubted, bronze answers "what did we actually pull?"
<code>silver/</code>	Cleaned and conformed: typed columns, parsed dates, normalized company names (legal entity → consumer brand), a documented duplicate strategy, nulls handled deliberately, one trustworthy record per complaint .	The messiness handled <i>here</i> , once, not re-handled ad hoc downstream. Standardize column names here too — different access paths name the same fields differently.
<code>gold/</code>	Analysis-ready tables the assistant queries. At minimum: complaint volume and timely-response rate by company × product × month . Also materialize whatever your RAG layer needs (e.g. the documented working set for embedding).	Correct, reconcilable numbers — gold aggregates should tie back to silver counts.

Each layer must be **rebuildable from the previous one** by re-running your code. Dedup decisions apply to retrieval/indexing; **analytical counts stay per-complaint** — every Complaint ID is one real complaint.

Field guide

Names below follow the bulk-download convention; the API names the same fields differently (e.g. `complaint_what_happened` for the narrative) — whatever the source calls them, standardize in silver.

#	Field	What it holds
1	<code>Date received</code>	Date the CFPB received the complaint. Arrives as a string — parse it.
2	<code>Product</code>	High-level product category (credit reporting, mortgage, checking account, ...).
3	<code>Sub-product</code>	Finer product classification; may be blank.
4	<code>Issue</code>	Standardized issue category the consumer selected.
5	<code>Sub-issue</code>	Finer issue classification; may be blank.
6	<code>Consumer complaint narrative</code>	The consumer's own free-text description. Present on every record in the pinned extract. PII redacted (see quirks).
7	<code>Company public response</code>	The company's optional public statement; often blank or boilerplate.
8	<code>Company</code>	The legal entity name of the company (e.g. "Block, Inc.", not "Cash App").
9	<code>State</code>	Two-letter US state/territory code; may be blank.
10	<code>ZIP code</code>	Consumer ZIP, frequently masked ("331XX") or blank.
11	<code>Tags</code>	Population tags such as "Older American" or "Servicemember"; may be blank.
12	<code>Submitted via</code>	Channel (Web, Referral, Phone, Postal mail, ...).
13	<code>Date sent to company</code>	Date the CFPB forwarded it to the company. String.
14	<code>Company response to consumer</code>	Resolution status ("Closed with explanation", "Closed with monetary relief", ...). Note: a category , not a dollar amount — the data holds no relief amounts.
15	<code>Timely response?</code>	Yes / No — did the company respond within the required window.

#	Field	What it holds
16	Complaint ID	Unique numeric identifier. Use this for citations.

Known quirks (handle them in silver)

- **PII redactions** — XXXX runs and {\$...} templated amounts inside narratives. The {\$...} figures are amounts the *consumer* mentions — **not** resolution amounts.
- **Near-duplicate narratives** — bulk-filed template complaints, heaviest in credit reporting. Each is still a distinct real complaint.
- **Legal names, not brands** — "Block, Inc." (Cash App), "TRANSUNION INTERMEDIATE HOLDINGS, INC." (TransUnion), "EARLY WARNING SERVICES, LLC" (Zelle), and so on.
- **Blank states/ZIPs, inconsistent casing, dates as strings** throughout.

The working set for semantic search

Embedding ~2M narratives is out of scope for a take-home. Materialize a **documented, reproducible working set** in gold for the RAG layer (tens of thousands is plenty), or use a retrieval strategy that narrows via metadata before any embedding happens. Your call — but write the choice down, make it reproducible, and mind the coverage warning in the brief: the fifteen questions probe specific companies, products, states, and months.

2. The fifteen questions

Shipped machine-readable as `questions.json ( id + question )`. Run every one through your assistant and send us its answers. They mix pure-narrative questions, questions with hard filters (company / product / state / date), and questions that need a number *and* the story behind it. **Some are unanswerable from this data by design** — the right response to those is a plain, grounded "*the data doesn't support this,*" not a guess. We won't tell you which ones.

ID	Question
q01	What are consumers most upset about when it comes to debt collection, and what kinds of experiences do they describe?
q02	What are the most common problems people reported with Cash App during 2025?
q03	Of the ten companies with the most complaints overall, which one is worst at responding on time — and what do consumers say happened in those late cases?
q04	Which banks saw the biggest surge in complaints during the March 2023 regional-banking collapse, and what were customers panicking about?
q05	When people try to get errors fixed on their credit reports, what do they say goes wrong with the process?
q06	What kinds of scams and fraud do consumers describe in complaints involving Zelle?
q07	Which company had the biggest spike in complaints in Q3 2025 compared to the previous quarter, and what was driving it?
q08	What are borrowers' biggest frustrations with MOHELA, based on what they write in their complaints?
q09	What are the most common complaints about State Farm's auto insurance claims process?
q10	What were California homeowners complaining about most in their mortgage complaints during the second half of 2025?
q11	When people complain about Cash App, how often does the company resolve it with actual relief versus just an explanation — and what do consumers describe in the cases that got no relief?
q12	How do people describe the impact on their lives when their bank suddenly freezes or closes their account?
q13	How is TransUnion handling identity-theft disputes, according to the people filing complaints?
q14	Have complaints about checking and savings accounts been going up or down across 2024 and 2025, and what's behind the change?
q15	For complaints that were closed with monetary relief, how much money did consumers typically get back?

3. What to send back

A small repository (or zip) plus a short write-up. Concretely:

1. **The pipeline and code**, organized so the medallion layers are visible as `bronze/ silver/ gold/` (folders, schemas, or both), with a **README** an engineer can follow to a running system in under ~30 minutes. Raw bronze data may be too large to commit — commit the **manifest and the fetch script** instead, and committed silver/gold artifacts (database file, vector index, cached embeddings) so we can skip re-pulling and re-embedding. Document both the quick path and a from-scratch rebuild. Never commit API keys; read them from the environment.
2. **The NL query harness (Part 3)** — runnable, so we can point it at your gold tables and ask questions in English at the demo. It must **generate and run its own query** (SQL or whatever your store speaks), execute **read-only**, **return the query it ran** with each answer, and fail gracefully when a question can't be expressed against the schema. A small set of natural-language → expected-result checks you can run is part of the evaluation (item 5), not a separate ask.
3. **A short decisions write-up** (≈2 pages): your source/access choice and why, the shape of your pipeline and solution, the trade-offs you made, what would break at 10× the data, and what you'd do next for production — including how a **daily refresh** would work against the daily-updating source (describe the approach — incremental, idempotent, monitored — don't build it).
4. **answers.json** — your system's answers to the fifteen questions, in the format below. For any answer whose number came from the query harness, include the generated query.
5. **Your evaluation** — the numbers you produced to check quality (retrieval, faithfulness, and NL→query correctness), and a few sentences on what they do and don't tell you.

answers.json format

An array of objects, one per question (IDs and text below are illustrative):

```
[
  {
    "question_id": "q07",
    "answer": "Complaint volume against Acme Bank rose from 610 to 940 (+54%) between Q2 and Q3 2025, the largest jump of any company; the narratives point to ...",
    "citations": [14396683, 14418746, 14509989],
    "query": "SELECT company, quarter, COUNT(*) ... GROUP BY company, quarter ...",
    "abstained": false
  },
  {
    "question_id": "q04",
    "answer": "The extract covers 2024–2025 only, so it can't speak to a March 2023 event ...",
    "citations": [],
    "query": null,
    "abstained": true
  }
]
```

- `question_id` — the ID from `questions.json`.
 - `answer` — your system's answer, verbatim. If a question is ambiguous, have it state the reading it took.
 - `citations` — the `Complaint ID` s that support the answer, as integers (typically a handful — cite the complaints that actually back each claim, not a bulk dump). Narrative claims need citations; pure aggregate figures should instead be reproducible from the query.
 - `query` — for any answer whose numbers came from the Part 3 harness, the query it generated and ran (a string). Use `null` or omit it for purely narrative answers.
 - `abstained` — `true` when the system declined the question for lack of grounding. It may still offer a clearly-labeled alternative; leave `citations` empty if nothing supports one.
-

4. Ground rules

- **Pull once, then freeze.** Snapshot your extract in bronze with a dated manifest and build everything on it. The source updates daily — we grade against your snapshot (that's what the manifest and the returned queries are for), so a live re-pull mid-project will only make your own numbers disagree with each other.
- **Any stack.** Any language, storage, orchestration, and any models — hosted or local. Justify the choices that matter.
- **AI assistance is fine** and expected; be ready to explain and defend every decision at the demo. Code you can't account for is the fastest way to come unstuck.
- **Costs are yours and should stay small** — the working-set approach (§1) keeps embedding costs to a few dollars hosted, or nothing local.
- **Scope honestly.** A few focused days, not the whole week. If you cut something, say so in the write-up — we read that as judgement, not as a gap.